Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

01. Both coroutines run on one event loop. What prints, and what is the smallest general fix for this shared in-process read-modify-write operation?

```
count = 0
async def increment():
    global count
    old = count
    await asyncio.sleep(0)
    count = old + 1
async def main():
    await asyncio.gather(increment(), increment())
    print(count)
asyncio.run(main())
```

A. 1; use a newly constructed asyncio.Lock inside each call.

B. 2; one event-loop thread makes the whole coroutine atomic.

C. 1; protect the whole read/await/write region with one shared asyncio.Lock.

D. 2; gather serializes mutations in argument order.

---

02. Three async evidence reads belong to one request. If any raises ValueError, unfinished siblings must be cancelled and awaited before control leaves the scope. They cooperate with cancellation. Which construct provides that behavior directly?

A. Create the three tasks inside an async with asyncio.TaskGroup() block.

B. Await asyncio.gather(*coroutines) with its default arguments.

C. Create three tasks and await only the first one to fail.

D. Use asyncio.gather(*coroutines, return_exceptions=True).

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

03. The request deadline cancels this coroutine. It releases a resource, then returns a cached answer. The contract requires cancellation to propagate to the caller. Which change is correct?

```
async def run():
    try:
        return await slow_call()
    except asyncio.CancelledError:
        await release()
        return cached_answer
```

A. Keep the code; catching CancelledError never changes task cancellation.

B. After release(), raise RuntimeError("cancelled") instead.

C. Wrap slow_call() in asyncio.shield() and keep returning the cached answer.

D. After release(), use bare raise in the CancelledError handler.

---

04. A request has a 2-second total budget. Retrieval takes 1.4 seconds and generation would take 1.2 more. Both are nonblocking, cancellation-cooperative coroutines. Which design enforces one shared deadline, allowing normal cleanup overhead?

A. Wrap each await in its own new asyncio.timeout(2) block.

B. Wrap both awaits in one asyncio.timeout(2) block and catch TimeoutError outside it.

C. Measure elapsed time only after both stages have completed.

D. Give retrieval 2 seconds, then give generation a fresh 1.2 seconds.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

05. A streaming source yields millions of fixed-size records faster than a worker can process them. The design must bound pending records in memory without dropping them. Which arrangement achieves that?

A. A bounded asyncio.Queue with create_task(queue.put(record)) for every incoming record.

B. A bounded asyncio.Queue, a fixed worker pool, and producers that await each queue.put().

C. An unbounded asyncio.Queue plus a semaphore around each worker call.

D. A list of one task per record, with a semaphore inside each task.

06. A synchronous SDK write has already started inside asyncio.to_thread(). The awaiting request task is cancelled. The SDK has no cancellation API. Which conclusion is justified?

A. to_thread converts cancellation into a successful SDK return value.

B. The thread is forcibly stopped, so the remote write cannot complete.

C. Cancellation rolls back any remote write made by the SDK.

D. The thread may finish the write; reconcile the operation outcome before assuming it did not happen.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

07. Use the default asyncio task factory and no explicit context argument. read_id() returns request_id.get(). What does the last line print, even if the child first runs after the parent sets B?

```
request_id = ContextVar("request_id")
async def read_id():
    return request_id.get()
async def main():
    request_id.set("A")
    child = asyncio.create_task(read_id())
    request_id.set("B")
    print(await child)
asyncio.run(main())
```

A. A, because create_task copies the current context at task creation.

B. B, because the child always reads the parent's latest ContextVar value.

C. None, because context variables are never inherited by child tasks.

D. A then B, because one read returns all values set during the request.

---

08. Four independent worker processes each create asyncio.Semaphore(8). Every outbound call holds one permit until it completes. All processes are busy. Ignoring other limits, how many calls can be in flight, and how could a fleet-wide cap of 8 be enforced?

A. Up to 4; each event loop can run only one outbound I/O operation.

B. Up to 8; asyncio semaphores automatically share permits across worker processes.

C. Up to 32; use coordinated shared admission control or fixed per-process budgets totaling 8.

D. Up to 32; rename each semaphore to the same global variable name.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

09. In ordinary GIL-enabled CPython 3.12, a profile shows four long independent pure-Python CPU transformations dominate latency. Data is picklable and work outweighs transfer overhead. Which option most directly enables use of multiple CPU cores?

A. Submit the transformations to a suitably sized ProcessPoolExecutor.

B. Wrap each transformation in async def and await asyncio.gather().

C. Run four asyncio.to_thread() calls and assume four-core Python execution.

D. Insert asyncio.sleep(0) repeatedly within each transformation.

10. stream() is an async generator with a finally block that releases a connection. The caller breaks after one useful item and requires cleanup before leaving this scope. Which pattern gives explicit deterministic closure?

A. Call rows.close() without awaiting anything after the loop.

B. Use a plain async for loop and assume break always immediately closes its generator.

C. Use async with contextlib.aclosing(stream()) as rows, then async for inside that block.

D. Wait for garbage collection to eventually run generator cleanup.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

11. The failure branch has status="error" and message:str. The success branch has status="ok" and value:float. Both forbid extra fields. Which contract rejects {"status":"ok","message":"timeout"} without accepting it as a failure event?

A. A dictionary[str, Any] plus a prompt asking the model for consistent fields.

B. One model with status:str, value:float | None=None, and message:str | None=None.

C. A validator that changes status to "error" whenever message is present.

D. A discriminated union of the two models, with Literal tags and discriminator="status".

12. The model accepts only positive integers. Under Pydantic 2 defaults, what prints after the shown update, and what is a safe boundary for this untrusted patch?

```
class Config(BaseModel):
    retries: int = Field(gt=0)
old = Config(retries=2)
new = old.model_copy(update={"retries": "bad"})
print(new.retries, type(new.retries).__name__)
```

A. A ValidationError; model_copy always validates updates.

B. bad str; merge into plain data and call Config.model_validate on the merged data.

C. 0 int; model_copy converts malformed integers to zero.

D. bad str; add deep=True to model_copy to enforce the field constraints.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

13. D declares a single field day: date. Both calls use strict=True. Which outcome is expected, and what does it imply for ingress tests?

```
class D(BaseModel):
    day: date
# Evaluate each call independently:
D.model_validate({"day": "2026-01-01"}, strict=True)
D.model_validate_json(
    '{"day":"2026-01-01"}', strict=True)
```

A. The dict call succeeds and JSON fails because JSON cannot represent dates at all.

B. Both fail because strict mode forbids every string representation in every input mode.

C. Both succeed because a YYYY-MM-DD string is always accepted for a strict date.

D. The Python-dict call fails and the JSON call succeeds; test the actual input mode used in production.

---

14. A reviewer treats this object as an immutable audit snapshot. What happens at append(), and which representation best fits an immutable collection of string tags?

```
class Snapshot(BaseModel):
    model_config = ConfigDict(frozen=True)
    tags: list[str]
s = Snapshot(tags=["reviewed"])
s.tags.append("changed")
```

A. Append succeeds; use tuple[str, ...] for tags as well as the frozen model.

B. Append raises ValidationError; frozen recursively freezes all fields.

C. Append succeeds; add extra="forbid" to make the list immutable.

D. Append creates a replacement model; the original audit snapshot is untouched.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

15. With default configuration, a model is mutated and then passed back to its own model_validate method. What happens, and which setting on Config specifically forces validation at that second boundary?

```
class Config(BaseModel):
    retries: int
cfg = Config(retries=2)
cfg.retries = "bad"
checked = Config.model_validate(cfg)
```

A. The second call raises; model_validate always rechecks an existing instance.

B. The invalid instance is accepted; set revalidate_instances="always" on Config.

C. The first assignment raises; assignment validation is enabled by default.

D. The invalid instance is accepted; set extra="forbid" on Config.

16. The public envelope is typed with User, but receives a UserWithToken instance. With default Pydantic 2 serialization and no custom serializers, what does model_dump() include?

```
class User(BaseModel):
    name: str
class UserWithToken(User):
    token: str
class Envelope(BaseModel):
    user: User
e = Envelope(user=UserWithToken(name="Jay", token="secret"))
print(e.model_dump())
```

A. Neither field; passing a subclass to a base-model field is invalid.

B. Both user.name and user.token; runtime subclasses always expose all fields.

C. Only user.name; fields defined only on the subclass are omitted.

D. Both fields, but token is automatically replaced with asterisks.

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

17. A hot path receives JSON arrays of already-defined Record models. It needs runtime validation and a list[Record] result, with no artificial wrapper field. Which design fits and avoids rebuilding the same validator per request?

A. Add a list[Record] return annotation to json.loads and trust the returned list.

B. Call typing.cast(list[Record], json.loads(raw)) per request.

C. Create TypeAdapter(list[Record]) once and call adapter.validate_json(raw) per request.

D. Create TypeAdapter(list[dict[str, Any]]) once and treat its results as Record instances.

---

18. An extracted label must belong to the request's supplied taxonomy, which varies across concurrent requests. A field validator reads info.context["labels"]. Which invocation supplies these rules without mutating process-global state?

A. Extraction(**payload, context={"labels": allowed_labels}) under default model configuration

B. Extraction.model_validate(payload, context={"labels": allowed_labels})

C. Extraction.model_validate({**payload, "context": {"labels": allowed_labels}})

D. Set Extraction.allowed_labels globally before calling Extraction.model_validate(payload).

Python-Pydantic-M-H-Set2

Medium-Hard | Concurrency and production contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

19. A monetary pipeline must preserve exact decimal input. Assume the default Decimal context and exact comparison. What prints, and what should change at ingestion?

```
a = Decimal(0.1 + 0.2)
b = Decimal("0.1") + Decimal("0.2")
print(a == Decimal("0.3"), b == Decimal("0.3"))
```

A. True False; arithmetic on decimal strings is less precise than float conversion.

B. True True; converting any float to Decimal recovers its intended decimal value.

C. False False; Decimal cannot exactly represent tenths.

D. False True; retain decimal text and construct Decimal before binary-float arithmetic.

---

20. V1 events carry amount_cents:int; V2 carries amount:Decimal and currency. The shared queue cannot route by consumer version. Old consumers reject V2, and deployment is rolling. Which rollout avoids sending unsupported events to old consumers?

A. First deploy consumers that validate both explicit versions and normalize them; confirm all readers are upgraded, then enable V2 producers.

B. Enable V2 producers first, then upgrade consumers when validation failures appear.

C. Rename amount_cents to amount in producers and let numeric coercion handle the semantic change.

D. Set extra="ignore" on old consumers and assume they can interpret new required fields.